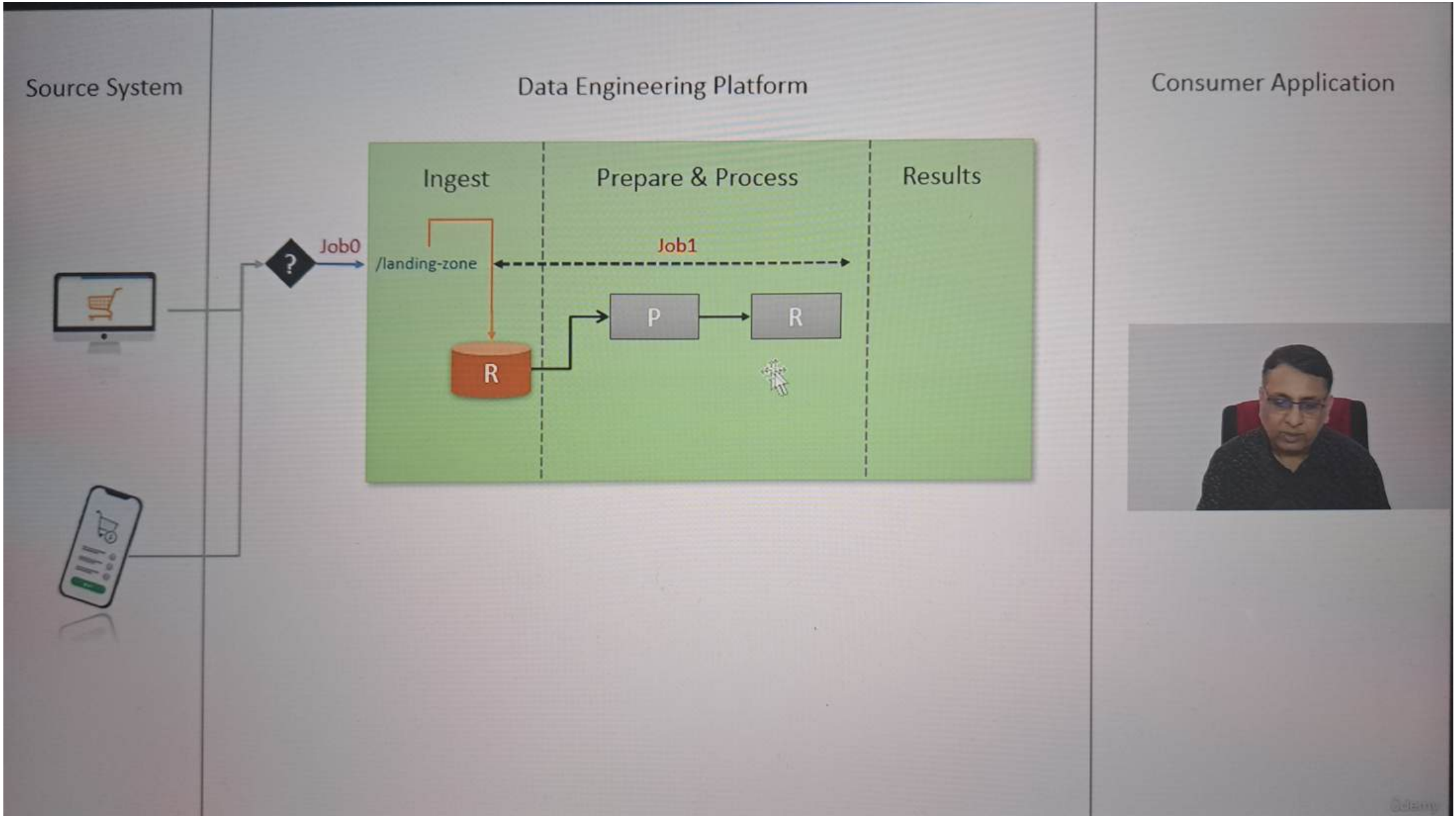
Source System
Data Engineering Platform
Consumer Application
Ingest
Prepare & Process
Results
Job0
Job1
/landing-zone
?
P
R
R

```
"InvoiceNumber": "8320594",
"CreatedTime": 1595688902254,
"StoreID": "STR7188",
"PosID": "POS825",
"CashierID": "OAS329",
"CustomerType": "PRIME",
"CustomerCardNo": "7051101351",
"TotalAmount": 5824.0,
"NumberOfItems": 3,
"PaymentMethod": "CASH",
"TaxableAmount": 5824.0,
"CGST": 145.6,
"SGST": 145.6,
"CESS": 7.28,
"DeliveryType": "HOME-DELIVERY",
"DeliveryAddress": {
    "AddressLine": "2465 Laoreet, Street",
    "City": "Dehri",
    "State": "Bihar",
    "PinCode": "637308",
    "ContactNumber": "2662305605"
},
"InvoiceLineItems": [
    {
        "ItemCode": "288",
        "ItemDescription": "Hutch",
        "ItemPrice": 1812.0,
        "ItemQty": 2,
        "TotalValue": 3624.0
    },
    {...
    },
    {...
    }
]
}
```


	InvoiceNumber	CreatedTime	StoreID	PosID	CustomerType	PaymentMethod	DeliveryType	City	State	PinCode	ItemCode	ItemDescription	ItemPrice	ItemQty	TotalValue
1	8320594	1595688902254	STR7188	POS825	PRIME	CASH	HOME-DELIVERY	Dehri	Bihar	637308	288	Hutch	1812	2	3624
2	8320594	1595688902254	STR7188	POS825	PRIME	CASH	HOME-DELIVERY	Dehri	Bihar	637308	558	Balloon clock	1633	1	1633
3	8320594	1595688902254	STR7188	POS825	PRIME	CASH	HOME-DELIVERY	Dehri	Bihar	637308	658	Chinois	567	1	567

```
    def getSchema(self):
        return """InvoiceNumber string, CreatedTime bigint, StoreID string, PosID string, CashierID string,
                  CustomerType string, CustomerCardNo string, TotalAmount double, NumberOfItems bigint,
                  PaymentMethod string, TaxableAmount double, CGST double, SGST double, CESS double,
                  DeliveryType string,
                  DeliveryAddress struct<AddressLine string, City string, ContactNumber string, PinCode string,
                  State string>
                  InvoiceLineItems array<struct<ItemCode string, ItemDescription string,
                      ItemPrice double, ItemQty bigint, TotalValue double>>
               """

    def readInvoices(self):
        return (spark.readStream
                    .format("json")
                    .schema(self.getSchema())
                    .load("{self.base_data_dir}/data/invoices")
                )
```

Shift+Enter to run
Shift+Ctrl+Enter to run selected text

03-invoice-stream Python

File Edit View Run Help Last edit was 10 minutes ago Provide feedback

Run all Connect Share Publish

```
    def getSchema(self):
        return """InvoiceNumber string, CreatedTime bigint, StoreID string, PosID string, CashierID string,
                  CustomerType string, CustomerCardNo string, TotalAmount double, NumberOfItems bigint,
                  PaymentMethod string, TaxableAmount double, CGST double, SGST double, CESS double,
                  DeliveryType string,
                  DeliveryAddress struct<AddressLine string, City string, ContactNumber string, PinCode string,
                  State string>
                  InvoiceLineItems array<struct<ItemCode string, ItemDescription string,
                      ItemPrice double, ItemQty bigint, TotalValue double>>
               """

    def readInvoices(self):
        return (spark.readStream
                    .format("json")
                    .schema(self.getSchema())
                    .load(f"{self.base_data_dir}/data/invoices")
                )
```

Shift+Enter to run
Shift+Ctrl+Enter to run selected text

```
        )

    def explodeInvoices(self, invoiceDF):
        return ( invoiceDF.selectExpr("InvoiceNumber", "CreatedTime", "StoreID", "PosID",
                                      "CustomerType", "PaymentMethod", "DeliveryType", "DeliveryAddress.City",
                                      "DeliveryAddress.State","DeliveryAddress.PinCode",
                                      "explode(InvoiceLineItems) as LineItem")
                               )

    def flattenInvoices(self, explodedDF):
        from pyspark.sql.functions import expr
        return( explodedDF.withColumn("ItemCode", expr("LineItem.ItemCode"))
                          .withColumn("ItemDescription", expr("LineItem.ItemDescription"))
                          .withColumn("ItemPrice", expr("LineItem.ItemPrice"))
                          .withColumn("ItemQty", expr("LineItem.ItemQty"))
                          .withColumn("TotalValue", expr("LineItem.TotalValue"))
                          .drop("LineItem")
              )
```

Shift+Enter to run
Shift+Ctrl+Enter to run selected text

```
                    .withColumn("TotalValue", expr("LineItem.TotalValue"))
                    .drop("LineItem")
                )

    def appendInvoices(self, flattenedDF):
        return (flattenedDF.writeStream
                    .format("delta")
                    .option("checkpointLocation", f"{self.base_data_dir}/chekpoint/invoices")
                    .optputMode("append")
                    .toTable("invoice_line_items")
        )
```

Shift+Enter to run
Shift+Ctrl+Enter to run selected text

```
        )

    def process(self):
        print(f"Starting Invoice Processing Stream...", end='')
        invoicesDF = self.readInvoices()
        explodedDF = self.explodeInvoices(invoicesDF)
        resultDF = self.flattenInvoices(explodedDF)
        sQuery = self.appendInvoices(resultDF)
        print("Done\n")
        return sQuery
```

Shift+Enter to run
Shift+Ctrl+Enter to run selected text

04-invoice-stream-test-suite Python

File Edit View Run Help Last edit was now Provide feedback

Run all | Connect | Share | Publish

Cmd 1

```
%run ./03-invoice-stream
```

Cmd 2

Python

```
class invoiceStreamTestSuite():
    def __init__(self):
        self.base_data_dir = "/FileStore/data_spark_streaming_scholarnest"

    def cleanTests(self):
        print(f"Starting Cleanup...", end='')
        spark.sql("drop table if exists invoice_line_items")
        dbutils.fs.rm("/user/hive/warehouse/invoice_line_items", True)

        dbutils.fs.rm(f"{self.base_data_dir}/chekpoint/invoices", True)
        dbutils.fs.rm(f"{self.base_data_dir}/data/invoices", True)

        dbutils.fs.mkdirs(f"{self.base_data_dir}/data/invoices")
        print("Done")

    def ingestData(self, itr):
        print(f"\tStarting Ingestion...", end='')
        dbutils.fs.cp(f"{self.base_data_dir}/datasets/invoices/invoices_{itr}.json", f"{self.base
```

04-invoice-stream-test-suite Python

File Edit View Run Help Last edit was 1 minute ago Provide feedback

Run all Connect Share Publish

Python

```
class invoiceStreamTestSuite():
    def __init__(self):
        self.base_data_dir = "/FileStore/data_spark_streaming_scholarnest"

    def cleanTests(self):
        print(f"Starting Cleanup...", end='')
        spark.sql("drop table if exists invoice_line_items")
        dbutils.fs.rm("/user/hive/warehouse/invoice_line_items", True)

        dbutils.fs.rm(f"{self.base_data_dir}/chekpoint/invoices", True)
        dbutils.fs.rm(f"{self.base_data_dir}/data/invoices", True)

        dbutils.fs.mkdirs(f"{self.base_data_dir}/data/invoices")
        print("Done")

    def ingestData(self, itr):
        print(f"\tStarting Ingestion...", end='')
        dbutils.fs.cp(f"{self.base_data_dir}/datasets/invoices/invoices_{itr}.json", f"{self.base
        invoices/")
        print("Done")

    def assertResult(self, expected_count):
        print(f"\tStarting validation...", end='')
        actual_count = spark.sql("select count(*) from invoice_line_items").collect()[0][0]
        assert expected_count == actual_count, f"Test failed! actual count is {actual_count}"
```

```
        print("Done")

    def assertResult(self, expected_count):
        print(f"\tStarting validation...", end='')
        actual_count = spark.sql("select count(*) from invoice_line_items").collect()[0][0]
        assert expected_count == actual_count, f"Test failed! actual count is {actual_count}"
        print("Done")

    def waitForMicroBatch(self, sleep=30):
        import time
        print(f"\tWaiting for {sleep} seconds...", end='')
        time.sleep(sleep)
        print("Done.")

    def runTests(self):
        self.cleanTests()
        iStream = invoiceStream()
        streamQuery = iStream.process()

        print("Testing first iteration of invoice stream...")
        self.ingestData(1)
        self.waitForMicroBatch()
        self.assertResult(1249)
        print("Validation passed.\n")

        print("Testing second iteration of invoice stream...")
```

04-invoice-stream-test-suite Python

File Edit View Run Help Last edit was 6 minutes ago Provide feedback

Run all Connect Share Publish

```
        streamQuery = iStream.process()

        print("Testing first iteration of invoice stream...")
        self.ingestData(1)
        self.waitForMicroBatch()
        self.assertResult(1249)
        print("Validation passed.\n")

        print("Testing second iteration of invoice stream...")
        self.ingestData(2)
        self.waitForMicroBatch()
        self.assertResult(2506)
        print("Validation passed.\n")

        print("Testing third iteration of invoice stream...")
        self.ingestData(3)
        self.waitForMicroBatch()
        self.assertResult(3990)
        print("Validation passed.\n")

        streamQuery.stop()
```

Shift+Enter to run

Shift+Ctrl+Enter to run selected text

04-invoice-stream-test-suite Python

File Edit View Run Help Last edit was 2 minutes ago Provide feedback

Interrupt my cluster Share Publish

Command took 0.09 seconds -- by prashantkumarpandey@gmail.com at 10/15/2023, 1:29:16 PM on my cluster

Cmd 3

Python

```
isTS = invoiceStreamTestSuite()
isTS.runTests()
```

Running command...

(1) Spark Jobs

```
Starting Cleanup...Done
Starting Invoice Processing Stream...
```

Shift+Enter to run

Shift+Ctrl+Enter to run selected text